

Plano — Nova ClinIA (v2) pelo pipeline completo, para comparar a implementação

Autor: Claude · **Data:** 2026-08-14 · **Status:** proposta para aprovação **Objetivo:** criar um **novo projeto ClinIA** (requisitos semelhantes) rodando o **pipeline INTEIRO do LangNet** — que agora tem as **features v3 embutidas** (Fases 0–7) — e **comparar** com a ClinIA original (`clinia-app5`), que foi construída **antes** da v3 (bugs corrigidos incrementalmente e features retrofitadas). Entregável: **report final didático em PDF**, etapa por etapa, com **cada prompt usado, cada tela e cada documento produzido**.

0. Hipótese a comprovar

Um app gerado **agora** (com a v3 no gerador) **já nasce** com: contrato de saída (A), contexto OKF (E), cadeia de comando (G), verificação (B), proveniência OKF (F), gate de requisito (C) e auditoria (D) — e **sem** a família de bugs que corrigimos manualmente na ClinIA original (`{raw}`), alucinação de tabela, FK nula, `enum↔float`, etc.). A comparação torna isso **visível e medível**.

1. Dimensões de comparação (o que vamos medir)

Dimensão	ClinIA original (v1)	ClinIA v2 (esperado)
Especificação	UCs sem constraints/edge_cases explícitos	UCs com constraints/edge_cases/verification + gap-analysis (reforço C)
tasks.yaml	output_schema/verification retrofitados via harness	nativos no code-gen
Código — contexto	injeção ad-hoc "DADOS DE ENTRADA"	bundle OKF <code>ws-server/knowledge/</code> nativo (E)
Código — saída	<code>{raw}</code> + remendos (<code>parseAgentResult/_cv</code>)	contrato <code>_coerce_to_schema</code> no <code>ws-server</code> (A)
Código — prompt	prompt plano	cadeia de comando em blocos (G)
Código — verificação	—	require_inputs/row_check/output_has (B)
Rastreabilidade	proveniência própria	OKF v0.2 <code>generated/verified</code> + Attested Computation (F)
Auditoria	—	<code>quality_report.md</code> + <code>assumptions.md</code> (C/D)
Bugs encontrados	9+ bugs + itens 1–4 (corrigidos ao longo de dias)	meta: 0 dessa família já na 1ª geração
Esforço de correção	alto (retrofit incremental)	baixo (features nativas)

2. Preparação (protocolo obrigatório, antes de começar)

- Commit + push** do estado atual; **backup** do banco `langnet` (`mysqldump --single-transaction`).
- Serviços no ar:** backend LangNet :8000 , frontend LangNet :3000 , **LLM** alcançável (`camerascasas.no-ip.info:1234`).
- Novo projeto** no LangNet (nome "ClinIA v2"), **mesmo seed de requisitos** da ClinIA original (`docs/clinica-medica/00-DESCRICA0-SISTEMA.md`) — para isolar a variável "versão do pipeline".

- 4. **Novo banco de dados do app** (`clinia_v2_ops`) separado do `clinia_ops` , para não misturar dados.
- 5. **Harness:** um `config.env` novo apontando para o app v2 (porta `ws/frontend` próprias) para regenerar/validar via `tools/regen/` .

Decisões a confirmar (assumo o padrão, você ajusta): (a) mesmo seed de requisitos [assumido: SIM]; (b) banco separado `clinia_v2_ops` [assumido: SIM]; (c) correções feitas **pelo LangNet** via refino na UI (não manuais), com screenshots [assumido: SIM, como você já pediu antes].

3. Etapas do pipeline — cada uma com prompt + tela + documento + verificação + captura

Para **cada** etapa vou registrar: (i) o **prompt exato** enviado ao LLM (capturado do `generation_log / chat_history` da sessão OU reconstruído chamando o `build_*_prompt` com os mesmos inputs; para refinós, a **instrução** que digitei), (ii) **screenshot da UI** do LangNet, (iii) o **documento/artefato** gerado, (iv) a **verificação** da etapa. Correções, se necessárias, **pelo agente do LangNet** (refino na UI).

#	Etapa (página LangNet)	O que capturar	Verificação da etapa
1	Criar projeto + Requisitos	prompt de análise/geração; tela; <code>requisitos.md</code>	doc tem seções e referências
2	Especificação	prompt (com reforço C); tela; <code>especificacao.md</code>	UCs têm constraints/edge_cases/verification (Fase 6)
3	Modelo de Dados	prompt; tela; <code>schema.sql/entities</code>	schema coerente; tabelas reais
4	Casos de Teste (CEG)	prompt; tela; casos	casos ligados aos UCs
5	UI Spec & Protótipo	prompt; tela; <code>ui_spec.json</code> + mockups	telas de negócio (cadastros+agênticas)
6	Sequência de Tarefas	prompt; tela; fluxo	ordem coerente
7	Agent-Task Spec	prompt (reforço C); tela; doc	tasks com constraints/edge_cases
8	Rede de Petri	prompt; tela; <code>petri.json</code>	lugares/transições/arcos
9	Geração de YAML	prompt; tela; <code>agents.yaml</code> + <code>tasks.yaml</code>	tasks ganham output_schema/verification + guard de coerência
10	Geração de Código	logs [CODE-GEN] ; tela; árvore do app	knowledge/ OKF , contrato no ws-server, cadeia G, verification, proveniência, <code>quality_report.md</code> , <code>assumptions.md</code>
11	Rodar o app + demo E2E	screenshots das telas; <code>smoke.sh</code>	cadeia clínica persiste (verify no banco) + red-team (injeção ignorada) + fault-injection (contrato)

Capturas do app gerado (Etapa 11): menu reorganizado, Recepção & Triagem (híbrida), Pré-diagnóstico, Seleção de Médico (dropdowns), Prontuário (cadeia herdada), Consulta — como fizemos na v1.

4. Correções (se necessárias) — pelo LangNet, via UI

Qualquer ajuste é feito **instruindo o agente do LangNet** (botão "Refinar com o agente" na etapa), **não** editando artefatos à mão. Cada correção entra no report com: a **instrução** (prompt), o **antes/depois** (histórico de versões)

e o **screenshot**. Se surgir um bug de gerador (não de conteúdo), corrijo no gerador com **checkpoint-antes** (mesmo protocolo das Fases 0–7) e registro.

5. Comparação final (v1 × v2)

Preencho a tabela da §1 com evidências reais dos **dois** apps (trechos de código, `knowledge/`, telas, resultados de smoke/red-team/fault-injection) e escrevo a análise: **o que a v3 trouxe nativamente e quais bugs deixaram de aparecer**.

6. Report final didático (PDF) — estrutura

1. **Capa + objetivo + hipótese**.
 2. **Seed de requisitos** (o pedido).
 3. **Por etapa (1–11):** *Prompt usado* (verbatim) → *Tela* (screenshot) → *Documento produzido* (trecho) → *Verificação* (resultado) → *Correções* (se houve, com antes/depois).
 4. **Comparação v1 × v2** (tabela + análise).
 5. **Provas E2E** (smoke VERDE, red-team, fault-injection) com telas.
 6. **Conclusão** (hipótese confirmada?) + ressalvas honestas. Tudo em **MD + PDF**, com screenshots embutidos.
-

7. Esforço, riscos e cronograma

- **Esforço:** alto. O pipeline inteiro roda no **LLM local** (cada etapa: minutos a dezenas de minutos; Especificação/UI-Spec/Código são as mais pesadas). Estimativa: **várias horas**, melhor em **blocos**.
 - **Riscos/mitigações:**
 - **Latência/flakiness do LLM** → retry; rodar etapas pesadas em background e monitorar.
 - **Backend :8000 trava** (hang recorrente) → restart via harness.
 - **Driver de UI** (Playwright token+proxy) pode exigir ajustes por etapa.
 - **Captura de prompt:** se o `generation_log` não guardar o prompt completo, reconstruo via os `build_*_prompt` (mesmos inputs) — registro qual método usei.
 - **Cronograma sugerido (blocos, com relatório parcial ao fim de cada):**
 - **Bloco A:** Etapas 1–4 (Requisitos → Especificação → Modelo de Dados → Casos de Teste).
 - **Bloco B:** Etapas 5–8 (UI Spec → Sequência → Agent-Task Spec → Petri).
 - **Bloco C:** Etapas 9–11 (YAML → Código → Rodar/demo/E2E).
 - **Bloco D:** Comparação v1×v2 + report final PDF.
-

8. Entregáveis

- **Novo app** `clinia-v2` (frontend + ws-server + `knowledge/` + `db/`), com banco `clinia_v2_ops`.
 - **Screenshots** por etapa (UI do LangNet) + telas do app gerado.
 - **Documentos** por etapa (requisitos, especificação, modelo, casos, ui_spec, sequência, agent-task, petri, yaml, código).
 - **Prompts** capturados por etapa.
 - **Report final** didático (MD + PDF) + a **tabela comparativa** v1×v2.
-

9. Critérios de aceitação

1. As 11 etapas executadas, cada uma com **prompt + tela + documento + verificação** no report.
2. O app v2 **roda** e o **fluxo clínico E2E persiste** (smoke VERDE) — com **red-team** e **fault-injection** passando.

3. O código v2 **contém nativamente** knowledge/ OKF, contrato no ws-server, cadeia de comando, verificação, proveniência OKF, `quality_report.md` e `assumptions.md`.
4. A **comparação v1×v2** documentada com evidências.
5. **Report final em PDF** entregue, didático e completo.

Ao aprovar, começo pelo **Bloco A** seguindo o protocolo (commit/push + backup antes), com relatório parcial ao fim de cada bloco — para você acompanhar e corrigir o rumo se precisar.